

Command	What it does	Typical usage
<code>mlflow --help</code>	Show all top-level commands and global options.	<code>mlflow --help</code>
<code>mlflow run <repo> [-no-conda]</code>	Execute a project from a local repo or remote URL. It will automatically install dependencies (via Conda, pip or Docker) and run the entry point.	<code>mlflow run .</code> (run current folder)
<code>mlflow run <repo> -P param1=val1 -P param2=val2</code>	Pass run-time parameters to the project.	<code>mlflow run . -P epochs=5</code>
<code>mlflow runs list</code>	List all runs in the active experiment (or a specified experiment).	<code>mlflow runs list --experiment-name my_exp</code>
<code>mlflow runs delete <run-id></code>	Delete a run (and its artifacts) from the tracking store.	<code>mlflow runs delete 123abc</code>
<code>mlflow experiments list</code>	List all experiments.	<code>mlflow experiments list</code>
<code>mlflow experiments create --experiment-name <name></code>	Create a new experiment.	<code>mlflow experiments create --experiment-name my_exp</code>
<code>mlflow experiments delete <exp-id></code>	Delete an experiment.	<code>mlflow experiments delete 1</code>
<code>mlflow artifacts list <run-id></code>	List all artifact files for a run.	<code>mlflow artifacts list 123abc</code>
<code>mlflow artifacts download --run-id <id> --path <artifact-path></code>	Download a specific artifact.	<code>mlflow artifacts download --run-id 123abc --path model.pkl</code>
<code>mlflow models serve --model-uri <uri> --host <ip> --port <port></code>	Start a REST endpoint that serves the model (uses <code>mlflow.pyfunc</code>).	<code>mlflow models serve --model-uri runs/123abc/model</code>
<code>mlflow ui --host <ip> --port <port></code>	Start the MLflow Tracking UI web server.	<code>mlflow ui --host 0.0.0.0 --port 5000</code>
<code>mlflow server --backend-store-uri <uri> --default-artifact-root <root></code>	Run a standalone MLflow Tracking server (HTTP).	<code>mlflow server --backend-store-uri sqlite:///mlruns.db --default-artifact-root s3://my-bucket/mlflow</code>
<code>mlflow.set_experiment("<name>")</code>	Create or activate an experiment.	<code>mlflow.set_experiment("Image_classification")</code>
<code>with mlflow.start_run():</code>	Context manager that automatically starts and ends a run.	<code>python
with mlflow.start_run():
mlflow.log_param("epochs", 10)
mlflow.log_param("learning_rate", 0.01)
mlflow.log_params({"lr":0.01, "batch_size":32})
mlflow.log_metric("accuracy", 0.95, step=epoch)
mlflow.log_metrics({"loss":0.3, "acc":0.92}, step=epoch)
mlflow.log_artifact("model.pkl")
mlflow.log_text("hello", "notes.txt")</code>
<code>mlflow.log_param(name, value)</code>	Log a single hyper-parameter.	
<code>mlflow.log_params(dict)</code>	Log many hyper-parameters at once.	
<code>mlflow.log_metric(name, value, step=None)</code>	Log a single metric (e.g., loss, accuracy).	
<code>mlflow.log_metrics(dict, step=None)</code>	Log many metrics.	
<code>mlflow.log_artifact(local_path, artifact_path=None)</code>	Upload a local file to the run's artifact folder.	
<code>mlflow.log_text(text, artifact_path)</code>	Log arbitrary text.	
<code>mlflow.log_model(model, artifact_path)</code>	Persist a model (sklearn, PyTorch, XGBoost, etc.) as an MLflow artifact.	<code>mlflow.sklearn.log_model(model, "model")</code>
<code>mlflow.register_model(artifact_uri, name)</code>	Register a model version in the Model Registry.	<code>mlflow.register_model("runs/123abc/model", "ImageClassifier")</code>
<code>mlflow.pyfunc.load_model(artifact_uri)</code>	Load a model via the generic pyfunc interface.	<code>model = mlflow.pyfunc.load_model("models/ImageClassifier/Production")</code>
<code>mlflow.pyfunc.run(artifact_uri, data)</code>	Run inference directly from the CLI (via <code>mlflow run</code>).	<code>pred = mlflow.pyfunc.run("models/ImageClassifier/Production", data)</code>
<code>mlflow.models.save_model(artifact_path, python_model=..., conda_env=None)</code>	Low-level helper to create a model package (used by <code>mlflow.log_model</code>).	<code>—</code>
<code>mlflow.projects.run(uri, parameters=None, use_conda=True)</code>	Programmatic equivalent of the CLI <code>mlflow run</code> .	<code>projects.run(".", parameters={"epochs":5})</code>
<code>mlflow.tracking.MlflowClient()</code>	Full-blown client for querying runs, experiments, registered models, etc.	<code>python
client = mlflow.tracking.MlflowClient()
runs = client.search_runs(experiment_ids=["1"])</code>
<code>client.get_run(run_id)</code>	Retrieve a run object (contains params, metrics, tags).	<code>run = client.get_run("123abc")</code>
<code>client.search_runs(experiment_ids, filter_string)</code>	Query runs with a SQL-like filter (e.g., <code>metrics.accuracy > 0.9</code>).	<code>client.search_runs(["1"], "metrics.loss < 0.2")</code>
<code>client.log_metric(run_id, key, value, step=None)</code>	Log a metric outside of a <code>with</code> block.	
<code>client.set_tag(run_id, key, value)</code>	Add a tag to a run.	<code>client.set_tag("123abc", "team", "mlops")</code>
<code>client.transition_model_version_stage(name, version, stage)</code>	Promote a model version (e.g., <code>Staging</code> → <code>Production</code>).	<code>client.transition_model_version_stage("ImageClassifier", 2, "Production")</code>